

2.4.2 Searching Algorithms: Linear Search

لینینر سرچ (Linear Search) کو ترتیب وار تلاش (Sequential Search) بھی کہتے ہیں۔

CS XI

Chapter 2: Computational Thinking & Algorithm

Lecture 10 of 13

2.4.2 Searching Algorithms: Linear Search

LEARN IT

Linear Search, also known as **sequential search**, works by examining each element in the list one by one, from the start, until the target is found (or the list ends). It works on both **sorted and unsorted data**.

A **flag** variable (here `ValueFound`) starts at 0 and becomes 1 when the item is found. After the loop, the flag tells us whether to print the item or "Not Found".



Algorithm (textbook)

```

Step 1: Start
Step 2: DATA := {8,3,6,1,7,2,4,5}
Step 3: N := 8
Step 4: FindValue := 7
Step 5: ValueFound := 0
Step 6: Repeat for PTR = 1 to N
Step 7:   If DATA[PTR] = FindValue:
           a. ValueFound = 1
           b. Goto Step 8
Step 8: If ValueFound = 1:
           Print DATA[PTR]
       Else:
           Print "Not Found"
Step 9: End
  
```

Pseudocode

```

Start
DATA = {8, 3, 6, 1, 7, 2, 4, 5}
N = 8
FindValue = 7
ValueFound = 0
For PTR = 0 to N-1
    If DATA[PTR] = FindValue then
        ValueFound = 1
        Exit Loop
    End If
End For
If ValueFound = 1 then
    Print DATA[PTR]
Else
    Print "Data Not Found"
End If
End
  
```

WORKED EXAMPLE

Worked example. Data list: 8, 3, 6, 1, 7, 2, 4, 5. Searching item: 7.

Step	Element checked	Condition (== 7)	Result
1	8	8 == 7 ?	Not found
2	3	3 == 7 ?	Not found
3	6	6 == 7 ?	Not found
4	1	1 == 7 ?	Not found
5	7	7 == 7 ?	Found! (Exit loop)

7 is found at position 5 (index 4) after 5 comparisons. The remaining items 2, 4, 5 are never checked.

If the item is missing — e.g., search for 9 — all 8 elements are checked, the flag stays 0, and "Data Not Found" is printed. This is the **worst case**: N comparisons.

WORKED EXAMPLE

Linear Search in Python

```
data = [8, 3, 6, 1, 7, 2, 4, 5]
find_value = 7
value_found = 0
for ptr in range(len(data)):
    print("Step", ptr + 1, ": checking", data[ptr])
    if data[ptr] == find_value:
        value_found = 1
        break
if value_found == 1:
    print("Found", data[ptr], "at position", ptr + 1)
else:
    print("Data Not Found")
```

OUTPUT

```
Step 1 : checking 8
Step 2 : checking 3
Step 3 : checking 6
Step 4 : checking 1
Step 5 : checking 7
Found 7 at position 5
```

break is Python's "Exit Loop".

Advantages and disadvantages

Advantages	Disadvantages
Does not care whether data is scrambled or sorted.	If the target is at the end of 1 million items, it does 1 million checks.
Extremely simple code (one loop, one if statement).	Time grows linearly (double the list = double the time).
Faster for very small lists (no setup time, such as sorting).	Inefficient if you need to search the same list thousands of times.
Does not require any extra space.	

REMEMBER IT

Linear Search is like looking for your friend in a queue by checking every face from the front. Best case: 1 check (first person). Worst case: N checks (last person or not there).

COMMON MISTAKE

Do not print "Not Found" inside the loop after every mismatch — that would print it many times. Decide only **after** the loop ends, using the flag. And remember to **exit the loop** once the item is found, or you waste comparisons.

اردو میں سمجھیں

لینیئر سرچ (Linear Search) کو ترتیب وار تلاش (Sequential Search) بھی کہتے ہیں۔ اس میں فہرست کے ہر عنصر کو شروع سے ایک ایک کر کے چیک کیا جاتا ہے جب تک مطلوبہ چیز نہ مل جائے۔ یہ ترتیب شدہ اور غیر ترتیب شدہ دونوں قسم کے ڈیٹا پر کام کرتی ہے اور بہت سادہ ہے، لیکن بڑی فہرست میں بہت وقت لیتی ہے۔

CHECK YOURSELF

1. What is another name for Linear Search?
2. In 8, 3, 6, 1, 7, 2, 4, 5, how many comparisons are needed to find 2?
3. What is the purpose of the variable ValueFound?

Answers: 1. Sequential search. 2. 6 comparisons (2 is in position 6). 3. It is a flag: 0 means not found yet, 1 means found; it decides what to print after the loop.

Next lecture: 2.4.2 Searching Algorithms: Binary Search